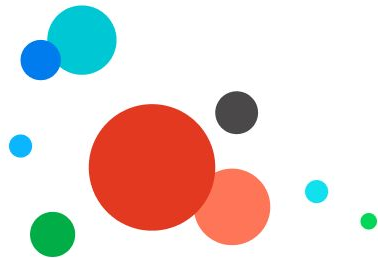




워크플로우 자동화의 세계

Apache Airflow 101

추영욱

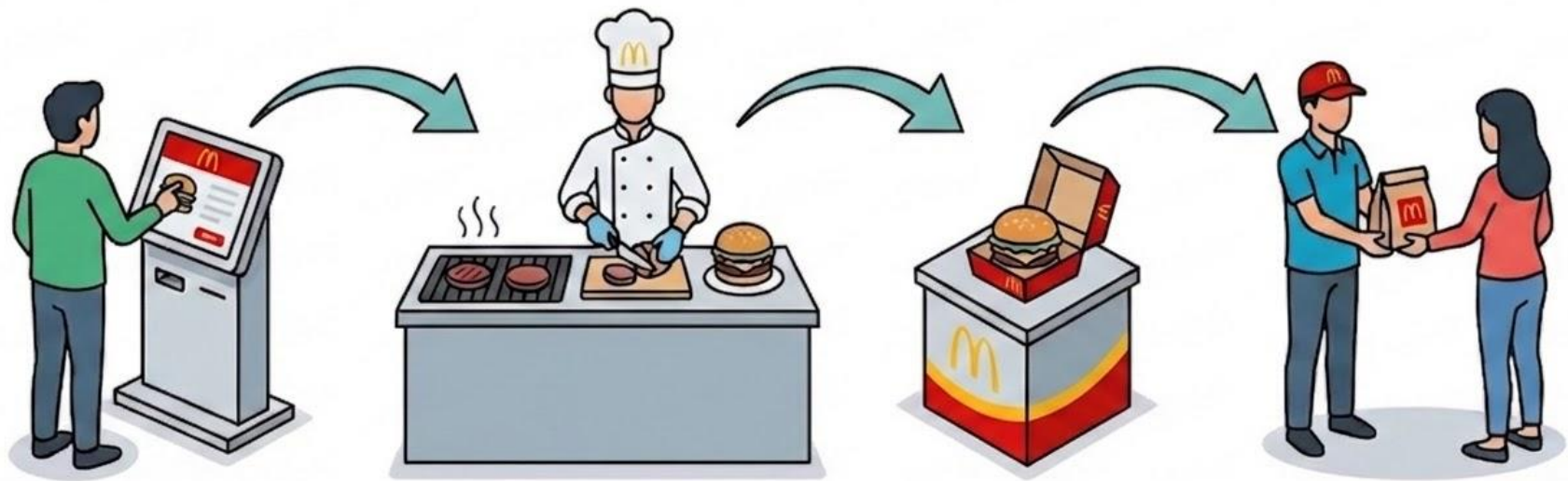


Airflow는 왜 생겼을까?

워크플로우를 관리하기 위해서



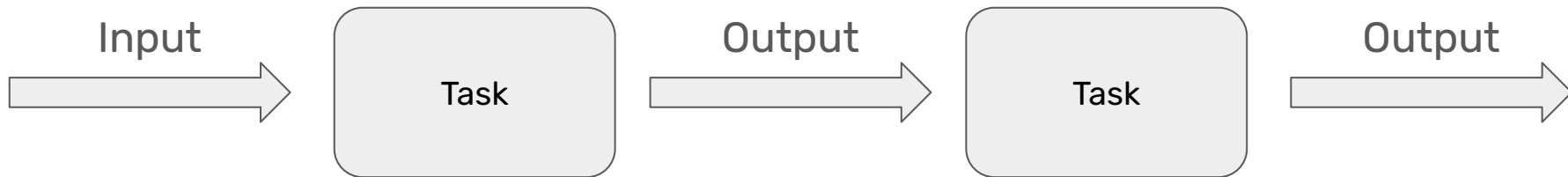
Workflow란? w/ 햄버거 주문





Workflow란?

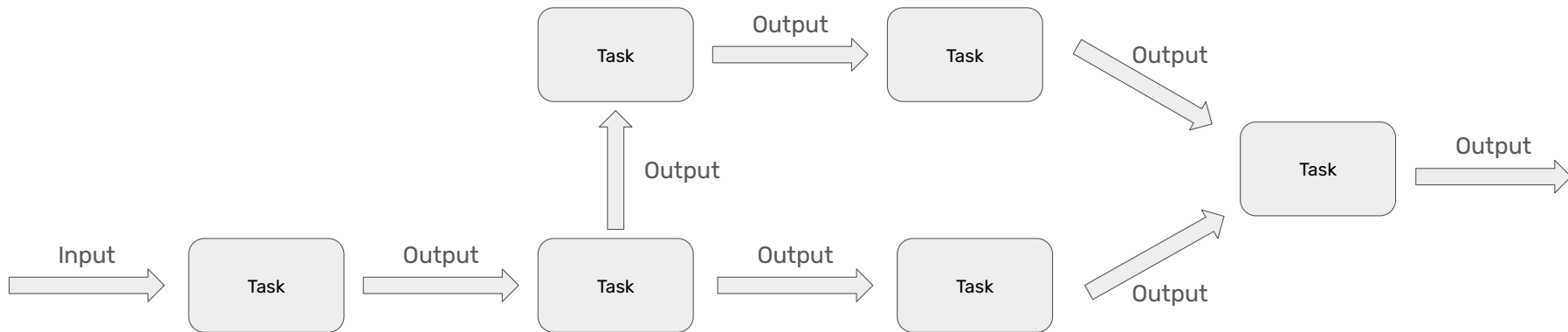
- 하나의 작업(Task)은 Input과 Output 으로 구성되어 있습니다.
- 이 작업들의 흐름을 워크플로우(Workflow)라고 합니다.

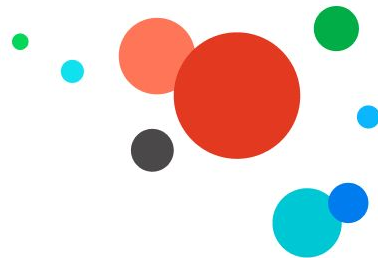




복잡해지는 워크플로우

- 현실 세계의 작업들을 연결하다보면 복잡해집니다.
- 워크플로우가 특정 주기로 실행되거나 특정 이벤트로 실행되어야하는 경우



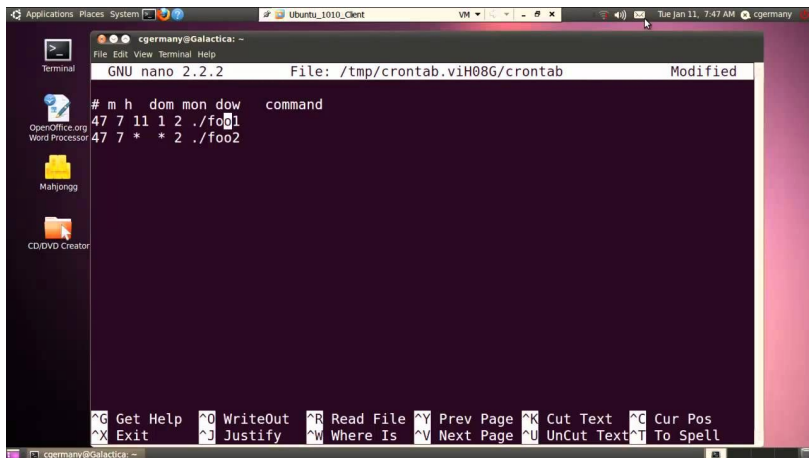


워크플로우 관리의 시작





이전에는 워크플로우를 어떻게 관리했는가?



```
# |----- minute (0 - 59)
# |----- hour (0 - 23)
# |----- day of the month (1 - 31)
# |----- month (1 - 12)
# |----- day of the week (0 - 6) (Sunday to Saturday;
# |                                     7 is also Sunday on some systems)
# * * * * * command to execute
```

```
$ crontab -e
```

```
$ * * * * * /bin/bash /path/to/your/script.sh
```




Cron을 이용하면서 생기는 몇가지 아쉬움

- 작업 등록의 복잡함
 - Cron에서 작업을 등록할 때는 파일의 경로, 실행 조건 등을 명령줄로 작성해야함.
- 과거 로그 파악
 - Cron에서 실행된 작업의 과거 로그를 확인하거나 저장하는 것이 매우 불편함.
 - 문제가 발생했을 때, 그 원인을 찾거나 실행 내역을 추적하는 것이 쉽지 않다.
- 복잡한 파이프라인을 만들 때 어려움
 - 여러작업이 서로 의존하는 복잡한 파이프라인을 Cron으로 구성하는 것은 매우 어려움

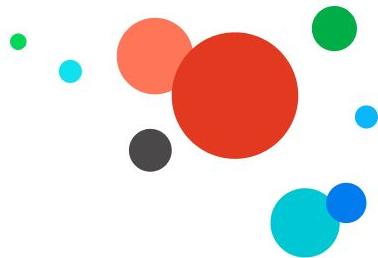
관리의 비효율성을 야기한다.

빠르게 성장하는 스타트업들에게 굉장히 큰 문제



워크플로우 오케스트레이션 도구의 필요성

1. 데이터의 복잡성 증가
 - a. 빅데이터 시대로 데이터의 복잡성 증가
2. 복잡한 워크플로우 제어
 - a. 여러 개의 작업이 연결되고 수행되는 복잡한 워크플로우에 대한 효율적인 관리 필요
3. 자동화, 모니터링 통합
 - a. 여러개의 작업을 배치, 이벤트로 스케줄링하고 이를 대시보드를 통해 모니터링 할 수 있어야 함
4. 안정적인 에러 로그 확인 및 재실행
 - a. 작업에서의 에러 로그 확인 및 재실행 가능해야 함.



여기서 등장하는 Airflow

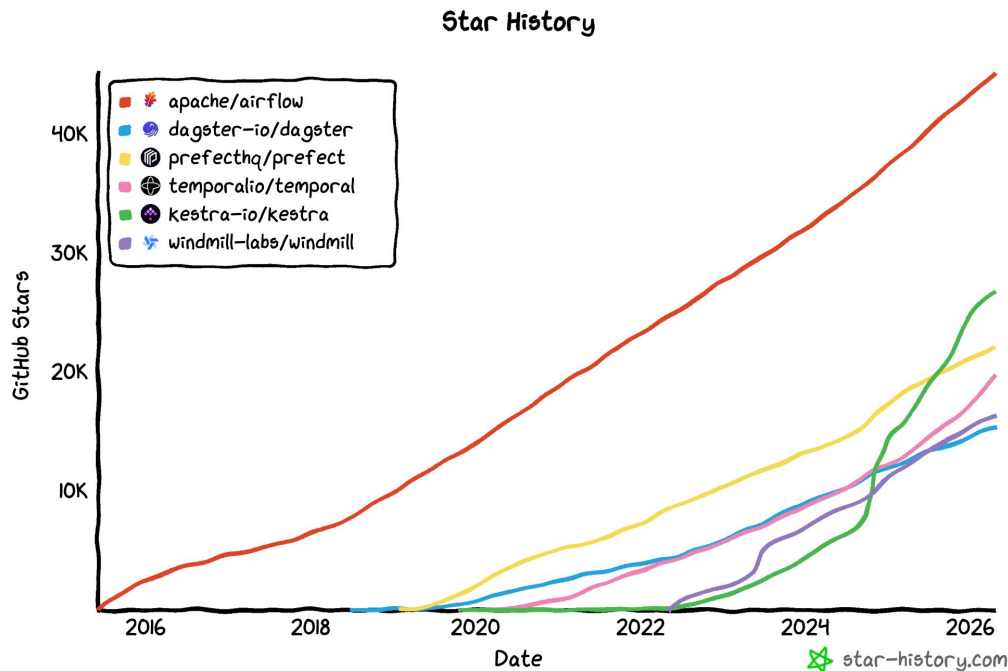




워크플로우를 프로그래밍 방식으로
작성, 스케줄링 및 모니터링 하기 위한
플랫폼



Why Airflow?



워크플로우 오케스트레이션 툴에서 최고의 위치를 10년 째 유지
중

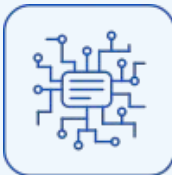


Airflow의 특징



Pure Python으로 작성

Python을 활용하여
워크플로우를 작성



Dynamic Data Pipelines

처리하는 데이터에 따라
워크플로우를 조정하는
동적 작업을 생성



Prebuilt Operators

일반적인 데이터
에코시스템에 연결할 수
있도록 **Operator**가 존재



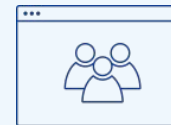
높은 확장성

필요한 만큼의 프로세스를
오케스트레이션 할 수
있음.



직관적인 UI

한눈에 작업들의 흐름을
볼 수 있는 UI



강력한 Community

강력한 커뮤니티를 통한
지속적인 기여 및 토론



Airflow는 Dag 라는 개념으로 워크플로우를 관리합니다.

Dag == Airflow의 워크플로우(Task들의 뭉치)



만약...

Dag를 어떤 수학적 개념인
Directed Acyclic Graph 라고 알고 계시다면...

Dag = Airflow의 워크플로우





Airflow가 워크플로우를 정의하는 법

```
@dag(  
    schedule=None,  
    start_date=pendulum.datetime(2021, 1, 1, tz="UTC"),  
    catchup=False,  
    tags=["example"],  
)  
def example_dag():  
    @task()  
    def extract(): ...  
  
    @task()  
    def transform(): ...  
  
    @task()  
    def load(): ...  
  
    extract() >> transform() >> load()  
  
example_dag()
```



Home



Dags



Assets



Browse



Admin



Docs



User

Dag

toy_chain_linear_vs_chain_complex



end_chain_linear_1

start_chain

start_chain_1

start_chain_linear

chain_linear_t0

chain_linear_t1

chain_linear_t2

chain_linear_t3

chain_linear_t4

chain_linear_t5

chain_linear_t6

chain_linear_t7

chain_linear_t8

chain_linear_t9

chain_t10

chain_t11

empty_1

end_chain_linear

chain_t12

chain_t13

chain_t14

chain_t15

chain_t16

chain_t17

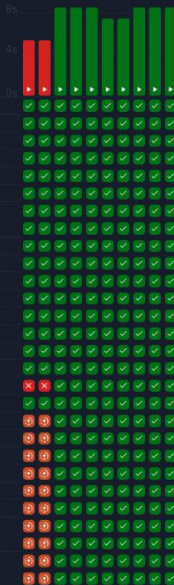
empty_2

chain_t18

chain_t19

end_chain

Options



toy_chain_linear_vs_chain_complex

Schedule

Latest Run

Next Run

Owner

Tags

Latest Dag Version

2025-09-23 17:19:01

airflow

toy, chain(), dependency_functions, +2 more

v5

Overview

Runs

Tasks

Calendar

Audit Log

Code

Details

Last 24 Hours

2025-09-22 17:19:43 - 2025-09-23 17:19:43

2 Failed Tasks

2 Failed Runs

Last 10 Dag Runs



14 Created Asset Events

Newest First

2025-09-23 17:15:36

asset://beta

Source: toy_chain_linear_vs_chain_complex

2025-09-23 17:15:35

asset://alpha

Source: toy_chain_linear_vs_chain_complex

2025-09-23 17:15:18

asset://beta

Source: toy_chain_linear_vs_chain_complex

2025-09-23 17:15:11

asset://beta

Source: toy_chain_linear_vs_chain_complex

2025-09-23 17:14:44

asset://beta

Source: toy_chain_linear_vs_chain_complex

2025-09-23 17:14:24

asset://alpha

Source: toy_chain_linear_vs_chain_complex

2025-09-23 16:57:38

asset://delta

Source: toy_chain_linear_vs_chain_complex

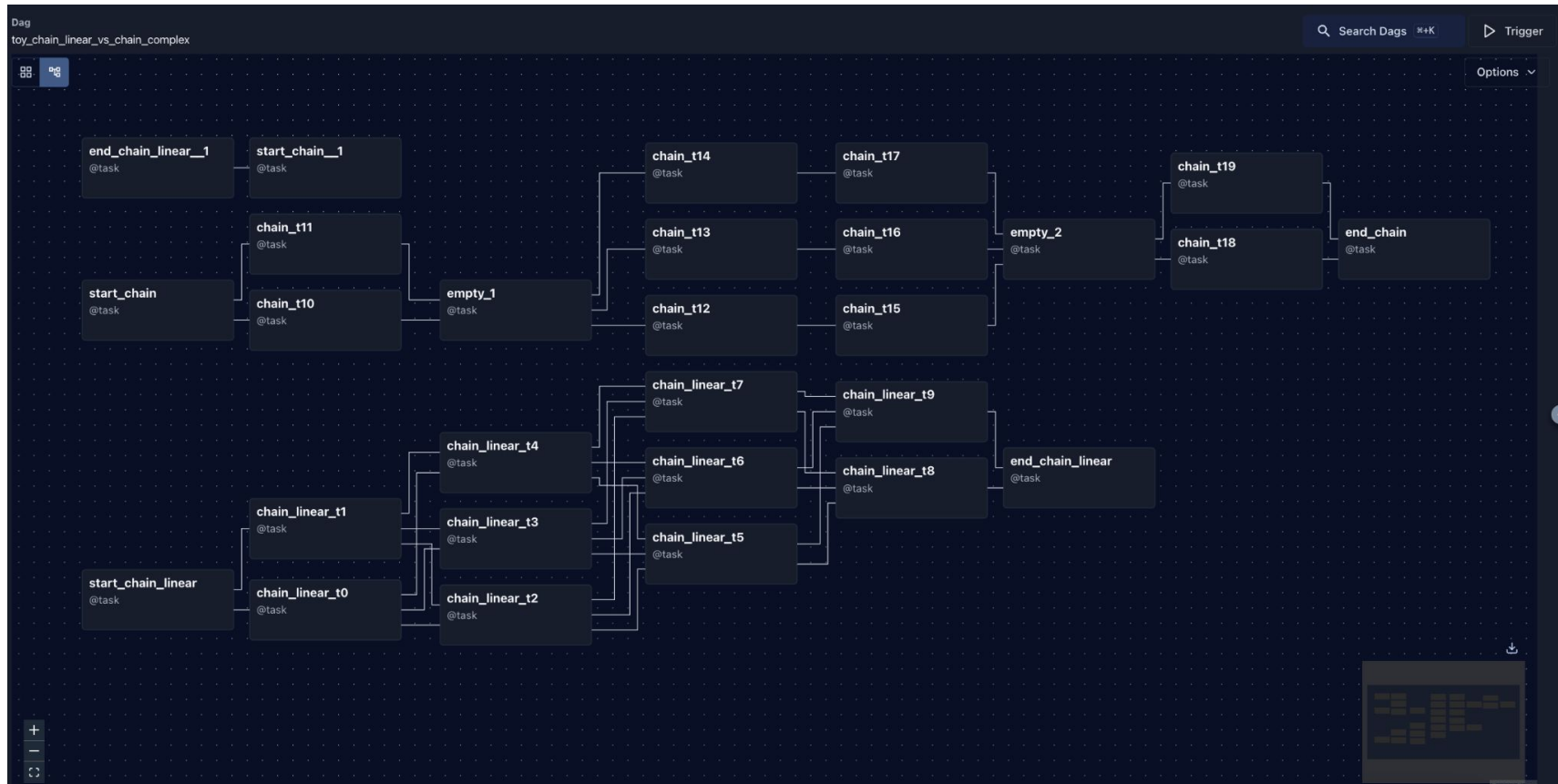
2025-09-23 16:57:36

asset://delta

Source: toy_chain_linear_vs_chain_complex



Airflow UI





Recap

- 작업(Task)은 input과 output으로 구성이 되어있습니다.
- 작업들의 흐름을 워크플로우라고 합니다.
- 워크플로우는 복잡해지고, 이를 관리할 수 있는 도구가 필요합니다.
- Airflow = 워크플로우를 프로그래밍 방식으로 관리하기 위한 플랫폼 입니다.
- Dag는 Airflow의 워크플로우로 작업들의 뭉치를 뜻합니다.
- Operator는 task의 템플릿.
- Provider는 Airflow로 연결할 수 있는 것들